

inCommon: migration prompts

Seven stages, in letter order. Paste each prompt into Claude Code as its own turn, attach the folder of the same letter, and do not start the next stage until the current one is done.

Every prompt is written to be pasted verbatim. Where a prompt asks you to decide something, it says so, and stopping to answer is the point rather than an interruption. Stage **g** is optional and can be skipped indefinitely.

Prompt	Attach	What it achieves
a	a. rules/	Repo exists, rules installed, nothing written yet
b	b. app/	The app runs locally, unmodified
c	c. verification/	206 assertions reproduced green, CI gate in place
d	d. data contracts/	Storage contract documented and guarded
e	e. deploy/	Bundle reproducible from source, one deploy path
f	f. design system/	Token drift detectable, colour rules enforced
g	g. stella unwired/	Optional: the AI guide, re-wired deliberately

Before stage a. Have the repository ready to receive files, and have a browser and any static file server available. There is nothing to install: no npm, no bundler, no framework. If that surprises you, read stage b before starting.

a. Orientation and the rules

STAGE A

Attach: **a. rules/** — PROJECT-RULES.md, inCommon-BRIEFING.md, canonical-build-report.md

I am migrating an existing application called inCommon into this repository. It was built in a design tool and is being handed to you complete and working. This first stage is orientation only: I do not want any application code written or changed yet.

Please do the following, in order:

1. Read all three attached documents in full. PROJECT-RULES.md is the project's own rules file, written over many iterations, and every rule in it was added because breaking it caused a real failure. Treat it as binding rather than advisory.
2. Rename PROJECT-RULES.md to CLAUDE.md and place it at the repository root, unchanged apart from removing the migration note at the top. You will read it automatically from there in future sessions.
3. Initialise the repository if it is not already: a sensible .gitignore, and an initial commit containing only these documents. Do not add application files yet, they arrive in stage b.
4. Then summarise back to me, in your own words and in under 400 words: what inCommon is, what the single-codebase rule means in practice, why styling is inline, what the 820px breakpoint does, and which colours may never be used as text. I want to see that the constraints landed before you touch the code.
5. Finally, list anything in those documents that strikes you as a risk, a contradiction, or a decision worth revisiting. Do not act on any of it. I will decide what changes and when.

Do not propose a framework, a build step, or a restructure in this stage. The app has no toolchain on purpose and the reasons are in the rules file.

b. The canonical build

STAGE B

Attach: **b. app/** — the app, support.js, 17 modules, the harness files, PWA assets, README.md

Here is the application itself. Read `b. app/README.md` first: it explains the file layout and the module load order.

The whole app is one file, `inCommonApp v2.dc.html`, about 12,600 lines: markup at the top, then a single ``class Component`` holding all the logic. Two shells live inside it, selected by viewport width at 820px. There is no build step and no compile. `support.js` is a vendored runtime that renders the markup and resolves the `{{ }}` holes: do not edit it.

For this stage:

1. Commit these files to the repository, unmodified. Do not reformat, do not reorder the module script tags, and do not "tidy" the single large file. The load order matters, several modules read globals set by earlier ones.

2. Get it running locally and tell me the exact command you used and the URL you opened. Serve it over http rather than opening it as a file, because the service worker and the web fonts want a real origin.

3. Confirm it boots by checking all of these, and report what you actually observed rather than what you expect:

- the splash clears and the Today screen renders
- the browser console is clean, with no errors
- resizing the window across 820px switches between the two shells
- the natal chart wheel draws on the Spirit screen
- navigating between Today, Spirit, Library, Throughline and Settings works

4. If anything fails to boot, diagnose and report before changing anything. The build was green when handed over, so a failure here is most likely a serving or a path problem rather than a code problem.

Do not refactor, split the file, extract components, convert to a framework, or change any styling in this stage. Getting it running unmodified is the entire goal, and it is also the baseline every later stage is measured against.

c. The verification harness

STAGE C

Attach: **c. verification/** — `README.md`, `run-tests-node.js`, `calculation-fixtures.json` (the harness itself is already in `b. app/`)

This stage is the most important one in the migration. The app ships with its own verification harness of 206 assertions, and it is the reason any future refactor of a 12,600 line file is safe to attempt at all. Read `c. verification/README.md` before starting.

The harness lives in `b. app/` rather than in the attached folder, because it loads the app by relative path and has to sit beside it: `handoff/tests-v14.js` plus the two Verification `*.dc.html` files.

For this stage:

1. Run the full harness by serving `b. app/` and opening `Verification V1.4.dc.html`. It starts on load and takes 25 minutes or more. Keep the tab in the foreground.
2. Report the result exactly: the total, the failures, and the per-group tallies. It was 206 of 206 passing at handover, in ten groups: A B C D S E F T R G. If your number is different, that difference is the first thing to understand, and do not proceed past it.
3. Then read the "Things that have bitten" section of the README and tell me which of those traps our CI setup would be vulnerable to.
4. Propose, but do not yet build, a CI arrangement. Constraints that matter:
 - The full browser harness takes 25+ minutes, so it cannot run on every commit. Suggest what runs per commit and what runs nightly or pre-release.
 - `run-tests-node.js` and `calculation-fixtures.json` check the calculation layer headlessly. That is the natural per-commit gate, and I would like it wired first.
 - Verification R - removal phase.dc.html runs one group in about a minute and is the fast smoke test.
5. Wait for my agreement on the plan before implementing it.

One rule I want you to hold to permanently, starting now: never make a red assertion pass by loosening the assertion. Several gates in this harness have been loosened before and every time it hid a real failure. If a check reports green, confirm it examined a plausible number of elements before believing it.

d. The data contract

STAGE D

Attach: **d. data contracts/** — `README.md`, `local-storage-schema.md`

This stage covers what inCommon writes to the device. Read both attached documents.

Everything the app knows lives in localStorage on one device: profiles, birth data, journals, moods, consent, theme choices. There is no account, no server and no sync, and that is a product promise rather than an unfinished feature.

For this stage:

1. Commit the schema documentation into the repository somewhere a future contributor will find it.
2. Verify the documentation against the code. Search the app file and profile-manager.js for every localStorage read and write, and tell me about any key that exists in the code but not in the schema, or in the schema but not the code. Do not fix drift you find, list it.
3. Treat the schema as a migration contract from here on. People have real journals in these keys. Write me a short policy, as a document in the repo, covering what has to happen before any key is renamed or any value shape changes: how existing data is migrated, how the migration is tested, and how a partial or failed migration is handled. Keep it under a page.
4. Flag the two keys the README calls out as easy to break, and explain in your own words what breaks if each is mishandled.

Do not change any storage key, any value shape, or any read or write path in this stage.

e. The deploy pipeline

STAGE E

Attach: **e. deploy/** — README.md, README-DEPLOY.md, index.html, sw.js, manifest, icons, Netlify config

This stage covers shipping. Read e. deploy/README.md and then README-DEPLOY.md, which is the full existing procedure.

The important distinction: index.html in this folder is a build output, not source. It is a single self-contained file of about 2MB with the runtime, all 17 content modules and the app inlined into it. The source is b. app/inCommonApp v2.dc.html. The bundle was produced by a tool in the design environment that does not exist here, so reproducing it is the work of this stage.

For this stage:

1. Commit the deploy folder as it stands, clearly marked as a build output.
2. Write a bundling script that reproduces index.html from b. app/. It needs to inline support.js, the 17 module scripts in their existing order, and the app itself, then add lang="en" to the tag. Keep it dependency-light and obvious rather than clever.
3. Prove the script works by generating a bundle and diffing it against the committed index.html. They will not match byte for byte, since a different tool produced the original, so instead confirm the generated bundle boots, has a clean console, and passes the fast smoke test from stage c.
4. Wire the cache rule in so it cannot be forgotten: the CACHE constant in sw.js has to change whenever index.html changes, and it must not change when only a test changes. It is at uncommon-v2.5 now. A check in CI is better than a line in a document.
5. The design project carried two deploy folders that had to stay byte-identical, which is a real hazard. Collapse to one in this repo and tell me what you removed.

Do not change the app source to make bundling easier without asking me first.

f. The design system

STAGE F

Attach: **f. design system/** – README.md, tokens.css, web-design-system.md

This stage covers colour and type. Read f. design system/README.md, then tokens.css and web-design-system.md, then re-read the "Colour rules that are not negotiable" and "The theme boundary" sections of CLAUDE.md.

Two things to understand before you touch anything. First, tokens.css is not loaded by the app and never has been: it is documentation. Styling is inline on every element and the tokens are CSS custom properties declared on three separate roots inside the app file. Second, a theme is a set of token deltas, never a style hole in the markup, because a hole cannot resolve while the page is still streaming and the app would sit unpainted until it finished.

For this stage:

1. Commit both documents.
2. Verify the three token roots against each other. CLAUDE.md lists exactly which tokens are allowed to differ between them and says the rest must match. Run that three-way comparison and report any token that has drifted. Several have drifted silently before.
3. Verify tokens.css against what the app actually declares, and report the differences. The documentation is the thing most likely to be stale.
4. Build me two automated checks, because both of these have failed in review and neither is catchable by eye:
 - A grep gate over the template zone of the app file. Between and the logic class, the only literal hexes allowed are #2fff8f, #8b5cf6, #e5534d, #000, #fff, and the three token-declaration roots themselves. Everything else must be a token. 186 literals were once missed here and stayed midnight-coloured under the other two themes.
 - A gate asserting #8b5cf6 is never used as a text colour. It measures about 4.2:1 on the app's own surfaces. Purple text is #b79bff. Grep the hex rather than the token name: a sweep for var(--ac2) once reported zero remaining while nine sites wrote the literal and kept failing.
5. Report what both gates find on the current build. I expect them to be clean, and if they are not I want to see it.

g. Stella, unwired

STAGE G · OPTIONAL, AND SAFE TO POSTPONE INDEFINITELY

Attach: **g. stella unwired/** — README.md, 6 dormant modules, stella-grounding.md, stella-voice-v144-reconciliation.md

This stage is optional and the app is complete without it. Do not begin it unless I have said I want Stella back.

Stella was inCommon's AI guide. Version 1.6.0 unwired it rather than deleting it: no module loads, no route reaches a chat, and `send()` returns with a debug note. The code was kept because the voice work in it is substantial. Read `g. stella unwired/README.md`, then the two documents, then the "What V1.6.0 removed" section of `CLAUDE.md`.

For this stage, in this order:

1. Commit the modules into the repository, clearly marked as dormant and not loaded by the build.
2. Map the seams before changing anything. There are three places in the app file: four script tags in the , the conjunction in `modulesReady()`, and `send()` with `flushOutbox()`. Show me each one as it stands today and describe exactly what re-wiring each would involve.
3. Note the ordering trap and confirm you have understood it: the script tags and the `modulesReady()` conjunction must be restored in the same change. `modulesReady()` gates the ready state, so if it waits for modules that were never loaded, the app never boots and the splash sits there forever.
4. Tell me what re-wiring would cost beyond the code. Stella needs a real API, which means a network call, which means the privacy claim the app makes on its own Settings screen changes. I consider that a product decision before an engineering one and I want it stated plainly, not buried.
5. Stop there and wait. Do not restore any script tag or change `modulesReady()` in this stage.

Two things that are not part of this and must not be swept up in it. The crisis controls, Get Help and the Yellow Pages and the directory, are safety features rather than AI: group R of the harness asserts they survived the removal and they must keep passing. And `checkVoice()` in `stella-voice-v144.js` enforces the project's zero-dash rule, building its detector from character codes so the file can obey the rule it enforces. A blind find-and-replace across this folder will silently disarm it.

After g

Once the stages are done the repository holds a running app, a green harness, a reproducible bundle and enforced colour rules. Reasonable next moves, roughly in order of value:

- **Self-host the three fonts.** They come from a CDN today, so the first load is not genuinely offline for an offline-first app.
- **Wire the headless calculation tests into CI** per commit, with the full browser harness nightly.

- **Only then discuss structure.** If splitting the single file ever becomes worth it, the harness is what makes it survivable, and the inline-style rule is what keeps the app painting instantly. Read the reasoning in CLAUDE.md before proposing either change.

The rule that outranks the rest. Never make a red assertion pass by loosening it, and never trust a green one you have not confirmed examined a plausible number of things. Both mistakes have shipped in this project and both were caught late.